

FAST-NUCES

National University of Computer and Emerging Sciences, Karachi

Operating Systems

CS-2006 | Instructor: Minhal Raza

FINAL PROJECT REPORT

myshell — Custom UNIX-Like Command Line Shell

Prepared By	
Student Name	Roll Number
Filza Tanweer	24k-0708
Fatima Hiader	24k-0551
Bismah Sheikh	24k-0795

Submission Date: May 10, 2026

GitHub: <https://github.com/FilzaTanweer/myshell>

Table of Contents

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives

4.1 Primary Objectives

4.2 Secondary Objectives

5. System Architecture and Design

5.1 High-Level Architecture

5.2 Backend: Read-Parse-Execute (REPL) Loop

5.3 Process Lifecycle

5.4 Signal Architecture

5.5 GUI Architecture and Integration

5.6 Frontend-Backend Communication Flow

6. Project File Structure

7. Detailed Module Explanations

7.1 src/myshell.c — Main Shell Logic

7.1.1 *print_banner()*

7.1.2 *install_signal_handlers()*

7.1.3 *handle_sigint()*

7.1.4 *handle_sigchld()*

7.1.5 *execute_command()*

7.1.6 *Built-in Commands (cd, pwd, help, history, exit)*

7.2 src/parser.c — Lexical Analysis (Phase 1)

7.3 src/history.c — Command History (Phase 3)

7.4 include/my_syscalls.h — Direct Kernel Syscalls (Phase 3)

8. GUI Implementation

8.1 Technology Choice: Python 3 + Tkinter

8.2 GUI Components

8.3 Subprocess and IPC Integration

8.4 ANSI Code Stripping

8.5 Executable Discovery

9. Operating System Concepts Used

10. Data Structures Used

10.1 cmd_t — Parsed Command Struct

10.2 history_node_t — Linked List Node

10.3 history_list_t — List Handle

10.4 String Array (args[])

11. Algorithms and Execution Flow

11.1 Complete Command Execution Flow

11.2 I/O Redirection Algorithm

11.3 History Eviction Algorithm

12. Compilation and Execution Instructions

12.1 System Requirements

12.2 Dependency Installation

12.3 Compilation

12.4 Running the Backend Shell

12.5 Running the GUI Frontend

12.6 Running the Test Suite

12.7 Memory Leak Check

13. Testing and Results

13.1 Test Suite Overview

13.2 Test Results by Phase

13.3 GUI Testing

14. Challenges Faced and Solutions

15. Comparison with Existing Shells

16. Work Distribution

17. Future Improvements

18. Conclusion

19. References

1. Abstract

This report presents the complete design, implementation, and evaluation of myshell — a custom, interactive, UNIX-like command-line shell developed in the C programming language for Ubuntu Linux. The project was undertaken as the final deliverable for the Operating Systems (CS-2006) course at FAST-NUCES Karachi, under the supervision of Instructor Minhal Raza.

myshell exposes and exercises the foundational mechanisms that every modern operating system provides: process creation through `fork()`, program loading through `execvp()`, parent-child synchronisation through `waitpid()`, file-descriptor manipulation for I/O redirection through `dup2()`, and asynchronous event handling through `sigaction()`. Crucially, three system call interfaces — `my_fork()`, `my_write()`, and `my_getpid()` — are implemented as direct kernel invocations using the Linux `syscall()` interface, thereby bypassing the standard glibc wrappers entirely.

The project is structured across five Work Breakdown Structure (WBS) phases: Lexical Analysis, Process Control, Built-in Commands and Low-Level Syscalls, Advanced Features (I/O redirection, signal handling, background execution), and Testing and Documentation. A lightweight Python/Tkinter graphical user interface (GUI) frontend complements the backend C shell, allowing commands to be submitted through a terminal-style window without altering any backend code. An automated shell test suite of 23 structured test cases validates correctness across all feature phases. The result is a fully functional, well-documented, and educationally rigorous operating system programming project.

2. Introduction

A command-line shell is the most direct interface between a user and the operating system kernel. Every command a user types is transformed by the shell into a sequence of system calls — the very same kernel primitives that govern process management, file access, and inter-process communication in any modern operating system. Yet, mainstream shells such as Bash, Zsh, and Fish abstract these low-level details so thoroughly that most users never observe them. This project deliberately dismantles those abstractions.

myshell is a purpose-built, lightweight shell developed entirely in C, designed to make every OS mechanism visible. Rather than wrapping functionality in convenience libraries, every core operation in myshell maps directly to a named system call or kernel interface. The fork-exec-wait lifecycle is implemented explicitly; file-descriptor duplication is performed step by step using `dup2()`; signal handlers are installed with the POSIX `sigaction()` API; and three system calls are invoked directly through the Linux `syscall()` instruction path, bypassing glibc wrappers entirely.

The shell supports a complete set of features including a `strtok()`-based lexical parser with enforced input constraints, five built-in commands implemented without `system()`, singly-linked-list command history, I/O redirection in three forms (truncate, append, and input), background process execution with automatic zombie prevention via `SIGCHLD`, and a coloured interactive prompt. A Python/Tkinter GUI frontend wraps the compiled shell executable as a subprocess, delivering a polished visual interface while leaving the C backend completely unmodified.

3. Problem Statement

Standard operating system interfaces are routinely hidden from end users by layers of abstraction. High-level shells process user input through complex parsing engines, script interpreters, and library wrappers that conceal the underlying system calls. As a result, computer science students may complete an operating systems course without ever directly observing the mechanics of process creation, address space duplication, signal delivery, or file descriptor manipulation.

This project addresses the following concrete problem: how can the full process lifecycle — from user input to program execution to process termination — be implemented at the lowest possible level in C, without relying on high-level abstractions, while remaining functionally complete and practically usable?

The solution requires bridging four distinct OS layers: user-space input handling, kernel-level process management, file descriptor-based I/O redirection, and asynchronous signal handling. Each layer must be implemented explicitly, documented transparently, and tested systematically. The project must also demonstrate that two otherwise separate concerns — a native C shell and a graphical user interface — can be integrated through standard inter-process communication mechanisms without compromising the integrity of the backend.

4. Objectives

4.1 Primary Objectives

- Implement a fully functional interactive UNIX-like shell in C that operates on a Read-Parse-Execute (REPL) loop.
- Demonstrate explicit use of `fork()`, `execvp()`, and `waitpid()` for process creation, image replacement, and parent-child synchronisation.
- Implement three direct kernel syscall wrappers — `my_fork()`, `my_write()`, and `my_getpid()` — using the Linux `syscall()` interface, bypassing glibc.
- Build a robust lexical parser using `strtok()` that enforces proposal constraints: a maximum input length of 100 characters and a maximum of 10 arguments.
- Implement five built-in commands — `cd`, `pwd`, `exit`, `history`, and `help` — without using the `system()` function.
- Support I/O redirection in three forms: output truncation (`>`), output append (`>>`), and input redirection (`<`) via `dup2()` and file descriptor manipulation.
- Handle POSIX signals correctly: intercept `SIGINT` (Ctrl+C) in the shell with `sigaction()` and `SA_RESTART`, and reap background children via a `SIGCHLD` handler using `WNOHANG`.
- Prevent zombie processes for background (`&`) jobs through the `SIGCHLD` handler.

4.2 Secondary Objectives

- Implement a singly-linked list with tail pointer for $O(1)$ command history management, supporting up to 50 entries with LRU eviction.
- Display an ANSI-coloured interactive prompt showing the current working directory, written via the `my_write()` syscall wrapper.
- Integrate a Python/Tkinter GUI frontend that launches the shell as a subprocess and communicates via piped `stdin/stdout` without modifying backend code.
- Build a Makefile-driven build system with targets for compilation, testing, memory checking, and cleanup.
- Provide an automated test suite (`run_tests.sh`) with 23 structured tests covering all five WBS phases.
- Produce complete project documentation suitable for academic submission and viva examination.

5. System Architecture and Design

5.1 High-Level Architecture

The myshell system consists of two distinct and independently functional layers: a native C backend shell and a Python/Tkinter GUI frontend. The two layers communicate through standard POSIX inter-process communication — piped file descriptors. The backend is entirely self-sufficient and can be used from any terminal; the frontend provides a graphical wrapper that does not alter backend behaviour in any way.

5.2 Backend: Read-Parse-Execute (REPL) Loop

The core execution model of myshell is the Read-Parse-Execute Loop (REPL), a classical shell architecture. The loop iterates indefinitely until an EOF (Ctrl+D) or the exit built-in is encountered:

```
1. print_prompt()      - Display coloured 'myshell:/cwd$' prompt via my_write()
2. fgets()             - Read up to MAX_INPUT (100) chars from stdin
3. history_add()        - Append raw input line to singly-linked history list
4. parse_line()         - Tokenise into cmd_t: args[], argc, infile, outfile, append,
                          background
5. dispatch            - Built-in? → execute directly. External? → execute_command()
6. execute_command()    - my_fork() → child: dup2 + execvp(). parent: waitpid() or
                          return
7. GOTO 1
```

5.3 Process Lifecycle

Every external command follows a strict five-step process lifecycle implemented in `execute_command()`:

1. `my_fork()` invokes `syscall(SYS_fork)` directly, creating a child process that is an exact duplicate of the parent's address space.
2. The child immediately restores default signal dispositions (`SIG_DFL`) and clears all signal masks, ensuring it behaves as an independent process.
3. If I/O redirection was parsed, the child calls `open()` to obtain a file descriptor and `dup2()` to replace `STDIN_FILENO` (0) or `STDOUT_FILENO` (1) with the file descriptor. The original fd is closed after duplication.
4. The child calls `execvp(cmd->args[0], cmd->args)`, replacing its process image with the target program. `execvp()` searches `$PATH` automatically. If it returns, execution failed.
5. The parent either calls `waitpid(pid, &status, 0)` for foreground commands (blocking until child exits), or returns immediately for background (&) commands — relying on the `SIGCHLD` handler to reap the child asynchronously.

5.4 Signal Architecture

Two signals are managed at the shell level using `sigaction()` with the `SA_RESTART` flag, which ensures interrupted system calls (such as `fgets()`) are automatically restarted rather than failing with `EINTR`:

SIGINT (signal 2, Ctrl+C): The shell installs `handle_sigint()`, which writes a newline via `my_write()` (async-signal-safe) and returns. The shell survives; child processes receive `SIGINT` via `SIG_DFL` and terminate normally.

SIGCHLD: The shell installs `handle_sigchld()`, which calls `waitpid(-1, NULL, WNOHANG)` in a loop, reaping all terminated background children immediately. This prevents zombie accumulation in the process table. `errno` is saved and restored because `waitpid()` modifies it.

5.5 GUI Architecture and Integration

The GUI frontend (`gui/gui.py`) is a Python 3 application built with the Tkinter standard library. It launches the compiled `myshell` binary as a subprocess using `subprocess.Popen` with piped `stdin`, `stdout`, and `stderr` (`stdout` and `stderr` are merged into a single stream):

```
self.process = subprocess.Popen(
    [self.shell_path],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    stderr=subprocess.STDOUT,
    bufsize=1, universal_newlines=True, text=True
)
```

A dedicated reader daemon thread continuously reads lines from the subprocess `stdout` and appends them to the GUI output widget. The thread operates independently of the Tkinter main event loop, preventing the GUI from blocking while waiting for shell output. A `threading.Lock` protects all writes to the Tkinter widget from concurrent access.

ANSI colour escape codes produced by the shell prompt are stripped by a compiled regular expression before display, ensuring clean text rendering in the Tkinter `ScrolledText` widget. Commands submitted via the GUI input field are written to the subprocess `stdin` pipe with a trailing newline, exactly as a human typing in a terminal would do.

5.6 Frontend-Backend Communication Flow

The complete communication sequence for a single command submitted via the GUI proceeds as follows:

6. User types a command in the Tkinter Entry widget and presses Enter or clicks Run.
7. `_run_command()` writes 'command\n' to `self.process.stdin` and calls `flush()`.

8. The myshell process, blocked in `fgets()`, receives the line on its `stdin`.
9. myshell parses and executes the command; all output goes to its `stdout`.
10. The reader thread (blocking on `self.process.stdout.readline()`) receives each output line.
11. The reader thread calls `_append_output()`, which strips ANSI codes and appends to the `ScrolledText` widget.
12. `root.update_idletasks()` refreshes the display without blocking the event loop.

6. Project File Structure

The project is organised into a clean, modular directory hierarchy separating source code, header files, tests, and GUI components:

File / Folder	Type	Purpose / Description
myshell_v2/	Root	Project root — contains Makefile, README.md
src/myshell.c	C Source	Main REPL loop, signal handlers, built-in commands, <code>execute_command()</code> with <code>fork-exec-wait</code>
src/parser.c	C Source	Phase 1 lexical analyser — <code>strtok()</code> tokenisation, redirection parsing, constraint enforcement
src/history.c	C Source	Singly-linked list command history — add, print, free with $O(1)$ append and LRU eviction
include/my_syscalls.h	C Header	Direct kernel syscall wrappers: <code>my_fork()</code> , <code>my_write()</code> , <code>my_getpid()</code> , <code>my_puts()</code>
include/parser.h	C Header	<code>cmd_t</code> struct definition, <code>MAX_INPUT=100</code> , <code>MAX_ARGS=10</code> constants, <code>parse_line()</code> prototype
include/history.h	C Header	<code>history_node_t</code> and <code>history_list_t</code> struct definitions, <code>HISTORY_MAX=50</code> , API prototypes
Makefile	Build System	Targets: <code>all</code> , <code>run</code> , <code>test</code> , <code>memcheck</code> , <code>clean</code> — gcc with <code>-Wall -Wextra -Wpedantic -g -std=c11</code>
tests/run_tests.sh	Test Suite	Automated Bash test suite — 23 tests across all five WBS phases
gui/gui.py	Python/Tkinter	GUI frontend — launches myshell subprocess, pipes stdin/stdout, scrolled terminal display
README.md	Documentation	Build instructions, feature table, OS concepts summary, viva Q&A quick reference

7. Detailed Module Explanations

7.1 src/myshell.c — Main Shell Logic

This is the entry point and control centre of the entire shell. It is responsible for the startup banner, signal handler installation, the REPL loop, built-in command dispatch, and the fork-exec-wait execution engine.

7.1.1 print_banner()

Called once at startup, this function uses `my_getpid()` to retrieve the shell's PID via a direct kernel syscall (`syscall(SYS_getpid)`), then uses `my_write()` to display the startup message. This serves as a live demonstration of Phase 3 direct syscall access.

7.1.2 install_signal_handlers()

Installs two signal handlers using the POSIX `sigaction()` API. The `SA_RESTART` flag is essential: it instructs the kernel to automatically restart system calls (such as `fgets()`) that were interrupted by signal delivery. Without this flag, `fgets()` would return `NULL` with `errno=EINTR` each time `Ctrl+C` was pressed, crashing the REPL loop.

7.1.3 handle_sigint()

The `SIGINT` handler for the shell process. It calls `my_write(STDOUT_FILENO, "\n", 1)` — a direct syscall — to print a newline and returns immediately. `my_write()` is used rather than `printf()` because `printf()` uses internal buffering locks (`flockfile/funlockfile`) that are not `async-signal-safe`: calling `printf()` from a signal handler can cause deadlocks if the signal arrives while `printf()` holds its lock.

7.1.4 handle_sigchld()

The `SIGCHLD` handler reaps terminated background children. It saves `errno`, calls `waitpid(-1, NULL, WNOHANG)` in a loop until no more children remain, then restores `errno`. The `WNOHANG` flag makes `waitpid()` non-blocking: if no children have exited yet, it returns immediately rather than blocking. `SA_NOCLDSTOP` prevents `SIGCHLD` delivery when children are merely stopped (via `Ctrl+Z`), avoiding unnecessary invocations.

7.1.5 execute_command()

The fork-exec-wait engine. This function receives a populated `cmd_t` and executes the external binary. The critical sequence in the child process after `my_fork()` is: restore signal defaults, clear signal masks, configure I/O redirection via `dup2()`, call `execvp()`. If `execvp()` returns, the binary was not found or could not be executed; `perror()` is called and the child exits with `EXIT_FAILURE`. The parent either waits synchronously (foreground) or prints a background PID message and returns (background).

7.1.6 Built-in Commands

Five built-ins are implemented as direct function calls in the parent shell process, never forking a child:

- `builtin_cd()`: calls `chdir(target)`. `cd` must be a built-in because `chdir()` only changes the calling process's working directory. A forked child's `chdir()` would leave the parent shell's directory unchanged.
- `builtin_pwd()`: calls `getcwd()` and writes the result via `my_write()`.
- `builtin_help()`: writes a formatted help text via `my_write()` listing all built-ins and syscall wrappers.
- `history`: calls `history_print()` on the global `history_list_t`.
- `exit`: writes "Goodbye!" via `my_write()` and breaks the REPL loop.

7.2 src/parser.c — Lexical Analysis (Phase 1)

`parse_line()` implements the Phase 1 lexical analyser. It receives a mutable character buffer (`strtok()` modifies it in-place) and populates a `cmd_t` structure. Tokens are extracted by `strtok()` using space, tab, and newline as delimiters.

The parser recognises five special token types: the background operator (`&`), output truncation (`>`), output append (`>>`), input redirection (`<`), and regular arguments. For each redirection operator, the immediately following token is consumed as the filename. If the filename is missing or is itself an operator, a syntax error is reported and the function returns -1. Regular argument tokens are stored in `cmd->args[cmd->argc++]` until `MAX_ARGS` (10) is reached. The `args` array is NULL-terminated so it can be passed directly to `execvp()`.

7.3 src/history.c — Command History (Phase 3)

The history module implements a singly-linked list with a tail pointer, directly satisfying the proposal requirement for a "struct `history_node`: A linked list or circular buffer for command history." The tail pointer enables $O(1)$ append complexity: new nodes are inserted directly at the tail without traversal.

When the list reaches `HISTORY_MAX` (50 entries), the oldest node (head) is freed and removed before the new node is appended. This provides LRU (least-recently-used) eviction semantics. Each node stores a one-based serial index, the command string (up to 100 characters), and a next pointer. `history_free()` traverses the entire list on shell exit, freeing every node to ensure zero memory leaks.

7.4 include/my_syscalls.h — Direct Kernel Syscalls (Phase 3)

This header defines four inline functions that bypass glibc and call the kernel directly via the Linux `syscall()` function. All three syscalls required by the proposal are implemented:

Function	Syscall Number (x86-64)	Linux Kernel Interface	Usage in myshell
my_write()	SYS_write = 1	write(fd, buf, count)	Prompt, banner, signal handlers, built-in output
my_getpid()	SYS_getpid = 39	getpid()	Startup banner PID display
my_fork()	SYS_fork = 57	fork()	execute_command() — every external command
my_puts()	(uses my_write)	write(1, s, strlen(s))	Convenience string wrapper

By using static inline functions, the compiler inlines the `syscall()` invocation at each call site, generating code that is functionally equivalent to the glibc wrapper but with no glibc intermediary. This is particularly important in signal handlers, where only async-signal-safe functions may be called.

8. GUI Implementation

8.1 Technology Choice: Python 3 + Tkinter

Tkinter was selected as the GUI framework because it is part of the Python 3 standard library, requiring no additional installation beyond Python itself. This aligns with the project's Ubuntu target environment where Python 3 is available by default. Tkinter provides a native-looking windowing system and a `ScrolledText` widget that closely mimics a terminal display. The choice avoids heavy GUI frameworks (PyQt, GTK) that would require separate package installation.

The GUI's role is strictly that of a presentation layer. It does not reimplement any shell logic, does not parse commands, and does not issue system calls. All computation remains within the compiled C shell binary, preserving the academic validity of the OS concepts demonstrated by the backend.

8.2 GUI Components

GUI Component	Description
Title Bar (tk.Frame)	Dark header (#2c3e50) with white label showing project title
Output Display (ScrolledText)	Dark terminal appearance (bg #1e1e1e, fg #00ff00, font Courier 10). Read-only; updated by reader thread. Auto-scrolls to latest output
Input Field (tk.Entry)	Single-line text input. Return key bound to <code>_run_command()</code> . Cleared after each submission
Run Button	Green (#27ae60) — submits the command to myshell stdin
Clear Button	Orange (#e67e22) — clears the output display area
Exit Button	Red (#e74c3c) — sends 'exit\n' to myshell, waits up to 2 seconds, then destroys the window
Status Label	Displays the resolved absolute path of the myshell executable

8.3 Subprocess and IPC Integration

The GUI launches myshell using `subprocess.Popen()` with `stdin=PIPE`, `stdout=PIPE`, and `stderr=STDOUT`. The `stdout` and `stderr` streams are merged so that both normal output and error messages (e.g., from `perror()`) appear in a single output stream. `text=True` and `universal_newlines=True` ensure all I/O is handled as decoded UTF-8 strings rather than raw bytes.

A daemon reader thread is started immediately after subprocess creation. It blocks on `self.process.stdout.readline()` — a blocking call that returns each time the child writes a complete newline-terminated line. Because this thread is a daemon, it is automatically terminated when the main application exits, preventing resource leaks.

All writes to the Tkinter ScrolledText widget are performed under a `threading.Lock` to prevent concurrent modification from both the reader thread and the main Tkinter event loop. The Tkinter state is toggled between `DISABLED` and `NORMAL` around each write to prevent the user from directly editing the output area.

8.4 ANSI Code Stripping

The myshell backend uses ANSI escape codes for its coloured prompt (green "myshell" + blue working directory). These codes are sequences of the form `ESC[...m`. Tkinter's ScrolledText widget does not interpret ANSI codes and would display them as raw characters. The GUI strips them using a pre-compiled regular expression:

```
self.ansi_escape = re.compile(r'\x1B(?:[@-Z\\-_]|\\[[0-?]*[ -/]*[@-~])')
text = self.ansi_escape.sub('', text)
```

8.5 Executable Discovery

At startup the GUI calls `_find_shell_executable()`, which checks three candidate paths in order: `./myshell` (current directory), `../myshell` (parent directory), and the absolute path of the myshell binary relative to `gui.py`'s location. The first executable found is used. If none is found, a modal error dialog is displayed and the application exits, prompting the user to run `make first`.

9. Operating System Concepts Used

OS Concept	Definition	Where Used in myshell
fork()	Creates a child process by duplicating the parent's address space (code, data, stack, heap, open file descriptors)	execute_command() — my_fork() wraps syscall(SYS_fork)
execvp()	Replaces the current process image with a new program binary, preserving PID but loading new text/data segments. Searches PATH automatically	execute_command() — child calls execvp() after fork
waitpid()	Parent waits for a specific child to terminate and collects exit status, removing the child from the process table	Foreground: waitpid(pid,&status,0). SIGCHLD handler: waitpid(-1,NULL,WNOHANG)
SIGINT	Signal 2 — delivered to foreground process group when user presses Ctrl+C	Shell ignores (custom handler); child inherits SIG_DFL and terminates
SIGCHLD	Signal 17 — delivered to parent when a child changes state (exits, stops, continues)	SIGCHLD handler reaps background children with WNOHANG
sigaction()	POSIX API for installing signal handlers with fine-grained control (SA_RESTART, SA_NOCLDSTOP, signal masking)	install_signal_handlers() in main()
SA_RESTART	Flag that tells the kernel to automatically restart slow system calls interrupted by a signal, rather than failing with EINTR	Applied to both SIGINT and SIGCHLD handlers
sigprocmask()	Modifies the calling thread's signal mask (blocked signals)	Child calls sigprocmask(SIG_SETMASK, &empty, NULL) to clear parent's mask
dup2()	Duplicates a file descriptor: dup2(fd, newfd) makes newfd refer to the same open file as fd	I/O redirection — makes STDIN/STDOUT point to redirect files
File Descriptors	Non-negative integers representing open files in the kernel's per-process file descriptor table	STDIN_FILENO(0), STDOUT_FILENO(1), redirect files
chdir()	Changes the calling process's working directory — affects only the calling process	builtin_cd() — must be a built-in for this reason
getcwd()	Returns the absolute pathname of the current working directory	builtin_pwd() and print_prompt()

syscall()	Linux glibc function that invokes a kernel system call by number, bypassing all glibc wrapper logic	my_fork(), my_write(), my_getpid()
Zombie Process	A terminated child whose exit status has not yet been collected by its parent — occupies an entry in the process table indefinitely	Prevented by waitpid() for foreground; SIGCHLD handler for background
Process Synchronisation	Coordination between concurrent processes to control execution order	Parent-child synchronisation via waitpid()
Background Execution	Running a process without blocking the shell — shell returns to prompt immediately while child continues executing	& operator — parent skips waitpid(); SIGCHLD reaps child

10. Data Structures Used

10.1 cmd_t — Parsed Command Struct

Defined in include/parser.h, cmd_t is the primary data exchange structure between the parser (parse_line()) and the executor (execute_command()) and built-in handlers). It carries all information derived from a single user input line:

```
typedef struct {
    char *args[MAX_ARGS + 1]; // NULL-terminated argument array (max 10 args)
    int  argc;                 // number of populated args entries
    int  background;           // 1 if trailing '&' was parsed
    char *outfile;             // filename after '>' or '>>' (or NULL)
    char *infile;              // filename after '<' (or NULL)
    int  append;               // 1 if '>>' (append mode), 0 if '>'
} cmd_t;
```

The args array is NULL-terminated (cmd->args[cmd->argc] = NULL) so it can be passed directly as the argv parameter to execvp() without any transformation.

10.2 history_node_t — Linked List Node

Each command entered by the user is stored in a heap-allocated history_node_t:

```
typedef struct history_node {
    int      index;           // 1-based command number
    char      cmd[MAX_CMD_LEN + 1]; // command string (max 100 chars +
    NUL)
    struct history_node *next; // pointer to next (newer) node
} history_node_t;
```

10.3 history_list_t — List Handle

The list handle carries pointers to both the head (oldest) and tail (newest) nodes, plus count and serial tracking fields. The dual-pointer design gives O(1) time complexity for both append and eviction:

```
typedef struct {
    history_node_t *head; // oldest entry — evicted when list is full
    history_node_t *tail; // most recent entry — new nodes appended here
    int             count; // current number of entries
    int             serial; // monotonically increasing command index
} history_list_t;
```

10.4 String Array (args[])

The `args[]` field of `cmd_t` is a fixed-size array of character pointers, bounded by `MAX_ARGS` (10). Each pointer points into the original input buffer (after `strtok()` substituted NUL terminators for delimiters). This design avoids any additional heap allocation for argument storage.

11. Algorithms and Execution Flow

11.1 Complete Command Execution Flow

The following trace follows the execution of a command such as "ls -l > output.txt &" from keyboard input to process termination:

13. User types 'ls -l > output.txt &' and presses Enter.
14. `fgets()` returns the line in the buffer. `strlen` is checked; if `> 100`, a warning is printed and `stdin` is drained.
15. `history_add()` copies the raw line to a new `history_node_t` allocated on the heap, appends it to the tail of `g_history`, increments `count` and `serial`.
16. `parse_line()` calls `strtok()` with "`\t\n`" delimiters. Tokens are extracted one by one: 'ls' → `args[0]`, '-l' → `args[1]`, '>' → sets outfile flag, 'output.txt' → `outfile = "output.txt"`, '&' → `background = 1`.
17. Dispatch logic: 'ls' is not a built-in. `execute_command(&cmd)` is called.
18. `execute_command()` calls `my_fork()` which invokes `syscall(SYS_fork)`. Returns child PID in parent, 0 in child.
19. Child process: `signal(SIGINT, SIG_DFL)`, `signal(SIGCHLD, SIG_DFL)`, `sigprocmask` clear all masks.
20. Child: `outfile != NULL`, so `open("output.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644)` → `fd`. `dup2(fd, STDOUT_FILENO)` makes `fd 1` point to `output.txt`. `close(fd)`.
21. Child: `execvp("ls", ["ls", "-l", NULL])`. Kernel searches `PATH`, loads `/bin/ls`. `ls` writes its listing to `fd 1`, which now points to `output.txt`.
22. Parent: `background == 1`, so `my_write()` prints '[PID] started in background'. Returns to REPL loop immediately.
23. When `ls` finishes, kernel delivers `SIGCHLD` to parent. `handle_sigchld()` calls `waitpid(-1, NULL, WNOHANG)`, reaping the child. No zombie remains.

11.2 I/O Redirection Algorithm

I/O redirection is achieved entirely through file descriptor manipulation in the child process after `fork()` but before `execvp()`. The algorithm for output redirection (`>`) is:

```
fd = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0644)
// fd is typically 3 (lowest available file descriptor)
dup2(fd, STDOUT_FILENO)
// STDOUT_FILENO (1) now refers to the same file as fd
// Any subsequent write to fd 1 writes to the file
close(fd)
// fd 3 is closed; only the duplicate (fd 1) remains
execvp(cmd, args)
// Loaded program writes to stdout (fd 1) → writes to file
```

11.3 History Eviction Algorithm

When `history_add()` is called and `count == HISTORY_MAX (50)`:

```
old = hl->head           // capture oldest node pointer
hl->head = old->next      // advance head to second node
if (hl->head == NULL)    // list had exactly one node
    hl->tail = NULL
free(old)                // release oldest node memory
hl->count--              // slot is now available
// then: allocate new node, append to tail, increment count
```

12. Compilation and Execution Instructions

12.1 System Requirements

Requirement	Value
Operating System	Ubuntu 22.04 LTS or Ubuntu 24.04 LTS (x86-64)
C Compiler	gcc (GCC 11 or later)
C Standard	C11 (-std=c11)
Python Version	Python 3.8 or later (for GUI frontend)
Tkinter	python3-tk (usually pre-installed on Ubuntu Desktop)
Build Tool	GNU Make
Optional	valgrind (for memory leak checking)
Optional	bash (v5+, for test suite execution)

12.2 Dependency Installation

```
# Install build tools (Ubuntu)
sudo apt-get update
sudo apt-get install gcc make valgrind python3 python3-tk
```

12.3 Compilation

```
# Navigate to project root
cd myshell_v2

# Build the shell
make

# Expected output:
# gcc -Wall -Wextra -Wpedantic -g -I./include -std=c11 -c src/myshell.c -o
src/myshell.o
# gcc -Wall -Wextra -Wpedantic -g -I./include -std=c11 -c src/parser.c -o
src/parser.o
# gcc -Wall -Wextra -Wpedantic -g -I./include -std=c11 -c src/history.c -o
src/history.o
# gcc -Wall -Wextra -Wpedantic -g -I./include -std=c11 -o myshell src/myshell.o
src/parser.o src/history.o
# Build successful -> ./myshell
```

12.4 Running the Backend Shell


```
# From the project root directory:
./myshell
# or
make run

# Expected startup banner:
# myshell -- Custom Unix Shell | FAST-NUCES OS Project
# Authors : Filza Tanweer, Fatima Haider, Bismah Sheikh
# Shell PID (via my_getpid syscall): 12345
# Type 'help' for commands. Type 'exit' to quit.
#
# myshell:/home/user$
```

12.5 Running the GUI Frontend

```
# Ensure myshell is compiled first (make), then:
cd myshell_v2
python3 gui/gui.py
```

The GUI window opens, automatically discovers the myshell binary, launches it as a subprocess, and displays the startup banner in the dark terminal area. Commands can be submitted via the Entry field.

12.6 Running the Test Suite

```
make test
# or directly:
bash tests/run_tests.sh

# Example output:
# =====
# myshell v2 -- Automated Test Suite
# =====
#
# Phase 3 -- Low-level syscall wrappers
# PASS Banner prints Shell PID via my_getpid()
#
# Phase 1 -- Lexical Analysis / Parsing
# PASS Empty line does not crash
# PASS Single command (echo hello_world)
# ... (23 tests total)
#
# Results: 23 passed, 0 failed / 23 total
```

12.7 Memory Leak Check

```
make memcheck
# Runs valgrind with --leak-check=full --track-origins=yes
# All heap memory allocated by history_add() is freed by history_free() on exit.
# Expected: 0 bytes in 0 blocks definitely lost.
```

13. Testing and Results

13.1 Test Suite Overview

The automated test suite (tests/run_tests.sh) contains 23 structured tests organised by WBS phase. Each test pipes a sequence of shell commands to myshell via its stdin, captures all output (stdout + stderr merged), and searches for an expected string using `grep -qF`. Tests are colour-coded: PASS in green, FAIL in red.

13.2 Test Results by Phase

Phase	Test Description	Expected Token
Phase 3 (Syscalls)	Banner prints Shell PID via <code>my_getpid()</code>	<code>my_getpid</code>
Phase 1	Empty line does not crash	Goodbye
Phase 1	Single command: <code>echo hello_world</code>	<code>hello_world</code>
Phase 1	Multi-arg: <code>echo alpha beta gamma</code>	<code>alpha beta gamma</code>
Phase 1	Too many args (>10) gives error	<code>too many arguments</code>
Phase 1	Missing file after > gives syntax error	<code>syntax error</code>
Phase 2	<code>ls</code> executes via <code>fork+execvp</code>	<code>myshell</code>
Phase 2	Absolute path <code>/bin/echo</code> works	<code>abs_path_ok</code>
Phase 2	Relative path <code>ls .</code> shows <code>myshell</code>	<code>myshell</code>
Phase 2	Invalid command: error not crash	<code>zzz_fake_cmd</code>
Phase 3	<code>pwd</code> prints directory	<code>/</code>
Phase 3	<code>cd /tmp</code> then <code>pwd</code> shows <code>/tmp</code>	<code>/tmp</code>
Phase 3	<code>cd</code> no arg goes <code>\$HOME</code>	<code>\$HOME</code> value
Phase 3	<code>exit</code> prints Goodbye	Goodbye
Phase 3	<code>help</code> lists built-ins	<code>cd</code>
Phase 3 (History)	history stores previous commands	<code>cmd_alpha</code>
Phase 3 (History)	history uses 1-based numbering	<code>1</code>
Phase 3 (History)	history preserves order	<code>bbb</code>
Phase 4	Output redirection (>) writes to file	<code>redir_output</code>
Phase 4	Input redirection (<) reads from file	<code>from_file_content</code>
Phase 4	Append redirection (>>) accumulates lines	<code>line1 + line2</code>

Phase 4	Background job starts and prints PID	background
Phase 4 (Constraint)	Input > 100 chars triggers warning	too long

All 23 automated tests pass on a correctly built Ubuntu 22.04 or 24.04 system. The signal handling test (Ctrl+C survival) is documented as a manual test in the test suite output, requiring interactive verification.

13.3 GUI Testing

GUI testing was performed manually by launching `gui/gui.py` after compilation, then submitting the following command sequence through the graphical interface:

- `echo hello from GUI` — verified output appears in ScrolledText widget.
- `pwd` — verified working directory displayed, ANSI codes stripped.
- `cd /tmp` and `pwd` — verified directory change reflected in output.
- `ls > /tmp/gui_test.txt` — verified file created with correct contents.
- `history` — verified previously entered commands appear in output.
- `exit` — verified shell terminates cleanly and GUI window closes.

14. Challenges Faced and Solutions

Challenge	Solution
Signal handler safety with printf()	Replaced all printf() calls in signal handlers with my_write() — a direct syscall() invocation guaranteed to be async-signal-safe
fgets() returning NULL on Ctrl+C without SA_RESTART	Added SA_RESTART flag to both sigaction() calls so interrupted system calls are restarted automatically by the kernel
Zombie processes from background jobs	Installed SIGCHLD handler that calls waitpid(-1, NULL, WNOHANG) in a loop, reaping all terminated children without blocking
Child process inheriting parent's signal customisations	Added explicit signal(SIGINT, SIG_DFL) and signal(SIGCHLD, SIG_DFL) in the child after fork, before execvp
cd not changing the shell's directory when run as external	Implemented cd as a built-in calling chdir() in the parent process directly, since chdir() only affects the calling process
ANSI escape codes displayed as raw text in Tkinter	Compiled a POSIX regex pattern and applied it to strip all ESC[...m sequences before inserting into ScrolledText
GUI freezing while waiting for shell output	Moved shell output reading into a dedicated daemon thread, leaving the Tkinter main thread free to handle events
Race condition on Tkinter widget from reader thread	Protected all widget modifications with threading.Lock and used root.update_idletasks() instead of update()
Enforcing 100-char input limit after fgets() truncation	Detected oversized input by checking strlen() of buffer; when the newline is absent and length equals MAX_INPUT, warning is printed and stdin is drained in a loop

15. Comparison with Existing Shells

Feature	myshell	Bash
Purpose	Educational — exposes OS internals	Production — full scripting environment
Lines of code	~500 C (excluding comments)	~200,000 lines
Process creation	my_fork() via syscall(SYS_fork)	fork() glibc wrapper
Syscall layer	Direct syscall() — no glibc	Standard glibc wrappers
I/O redirection	> >> < via dup2()	Full redirection suite + here-docs
Signal handling	SIGINT + SIGCHLD via sigaction()	Comprehensive signal management
Built-in commands	5 (cd, pwd, exit, history, help)	60+ built-ins
Scripting	Not supported (by design)	Full if/else/for/while/functions
Job control	Background (&) with SIGCHLD reaping	fg, bg, jobs, process groups
Globbing	Not supported	Full glob expansion
Piping ()	Not supported	Full pipe chains
Command history	Singly-linked list, 50 entries	Configurable, disk-persisted
GUI frontend	Python/Tkinter subprocess wrapper	No native GUI
Coloured prompt	ANSI via my_write()	Fully customisable PS1
Transparency	Every syscall is visible and named	Abstractions hide OS layer

16. Work Distribution

Team Member	Role	Responsibilities
Filza Tanweer (24k-0708)	Lead Developer — Process Management	Primary author of <code>src/myshell.c</code> : REPL loop, <code>execute_command()</code> fork-exec-wait engine, signal handler installation, <code>print_banner()</code> . Author of <code>include/my_syscalls.h</code> : direct kernel syscall wrappers (<code>my_fork</code> , <code>my_write</code> , <code>my_getpid</code>). Integration testing and GitHub repository management.
Fatima Hiader (24k-0551)	Parser Architect — Lexical Analysis	Primary author of <code>src/parser.c</code> and <code>include/parser.h</code> : <code>strtok()</code> tokeniser, redirection parsing (<code>></code> , <code>>></code> , <code><</code>), constraint enforcement (100-char limit, 10-arg limit), <code>cmd_t</code> struct design. Author of <code>src/history.c</code> and <code>include/history.h</code> : singly-linked list history implementation with $O(1)$ append and LRU eviction.
Bismah Sheikh (24k-0795)	Systems & UI — Integration, GUI, Testing	Primary author of <code>gui/gui.py</code> : Python/Tkinter GUI frontend, subprocess integration, reader thread, ANSI stripping, thread-safe output display. Author of <code>tests/run_tests.sh</code> : 23-test automated test suite. Final documentation, README, and project report preparation.

17. Future Improvements

24. Pipe Support (|): Implement command pipelines using `pipe()` to create a shared pipe file descriptor pair between two child processes, connecting the first child's stdout to the second child's stdin via `dup2()`.
25. Command Auto-Completion: Implement Tab-completion using `readline()` or a custom key handler that scans the PATH directories for matching binary names.
26. Job Control (fg, bg, jobs): Add full POSIX job control using process groups (`setpgpgrp()`), the SIGTSTP signal (Ctrl+Z), and the `tcsetpgrp()` API to transfer terminal control between shell and foreground job.
27. Persistent Command History: Save the history linked list to a `~/.myshell_history` file on exit and reload it on startup, using the existing history API with minor additions.
28. Here-Document Support (<<): Implement here-document input redirection by reading lines from stdin until a terminator token is found, storing them in a temporary file, and redirecting the command's stdin from that file.
29. Environment Variable Expansion: Add support for \$VAR substitution in arguments by scanning each parsed token for '\$' prefixes and replacing with `getenv()` results.
30. GUI Enhancements: Add command history navigation in the GUI using Up/Down arrow keys, colour-coded output (red for errors, green for success), and a session log export feature.

18. Conclusion

myshell successfully achieves all primary and secondary objectives defined in the project proposal and the five-phase Work Breakdown Structure. The project delivers a fully functional, interactive UNIX-like shell implemented in C that explicitly demonstrates every major operating system mechanism studied in the CS-2006 Operating Systems course.

Every `fork()`, `execvp()`, `waitpid()`, `dup2()`, `sigaction()`, and `chdir()` call is intentional, named, and documented. Three kernel system calls — `my_fork()`, `my_write()`, and `my_getpid()` — are invoked through the direct Linux `syscall()` interface, bypassing the standard C library entirely and demonstrating a clear understanding of the boundary between user space and kernel space. The lexical parser enforces the proposal-specified constraints of 100-character input and 10-argument maximum. The command history uses a singly-linked list with tail pointer as the proposal required. Zombie processes are actively prevented through a correctly implemented `SIGCHLD` handler. All five built-in commands are implemented without the `system()` function.

The Python/Tkinter GUI frontend demonstrates how a graphical application can integrate with a native OS-level program through standard inter-process communication — file descriptor pipes — without modifying the backend. This integration exemplifies a fundamental software architecture principle: separation of concerns. The 23-test automated suite provides reproducible evidence of correctness across all implemented features.

This project has provided the team with hands-on experience in systems programming, direct kernel interaction, multi-process coordination, and the practical application of POSIX APIs. The result is a system that is not merely functional but educationally transparent — every line of code corresponds to a named OS concept.

19. References

31. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
32. Kerrisk, M. (2010). The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press.
33. Linux man-pages Project. (2024). fork(2), execvp(3), waitpid(2), sigaction(2), dup2(2), syscall(2), chdir(2), signal(7). Linux manual pages. <https://man7.org/linux/man-pages/>
34. GNU Project. (2024). The GNU C Library (glibc) Reference Manual. Free Software Foundation. <https://www.gnu.org/software/libc/manual/>
35. IEEE. (2018). POSIX.1-2017: The Open Group Base Specifications Issue 7, 2018 Edition. IEEE Std 1003.1-2017. IEEE/The Open Group.
36. Stevens, W. R., & Rago, S. A. (2013). Advanced Programming in the UNIX Environment (3rd ed.). Addison-Wesley Professional.
37. Python Software Foundation. (2024). subprocess — Subprocess management. Python 3.12 Documentation. <https://docs.python.org/3/library/subprocess.html>
38. Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Python 3.12 Documentation. <https://docs.python.org/3/library/tkinter.html>
39. FAST-NUCES CS-2006 Operating Systems Course Materials. (2026). Lecture slides and lab handouts. Instructor: Minhal Raza.
40. Project Source Repository: <https://github.com/FilzaTanweer/myshell>